AI Course Recommender & Career Roadmap Generator

A 7-week data-science project. You will build, compare, and evaluate a course-recommendation engine that maps a person's skill gaps to the right courses in the right order — then add an AI-written roadmap on top.

`5 students · 1 team` `Mentor: Dev Dutta` `Kickoff: Fri 19 Jun 2026`
`Report due: 31 Jul 2026` `Meetings: Thu 7–9 PM IST · Teams`

01

The project, in one page

The problem. Someone with a career goal ("I want to become a Data Scientist") rarely knows which skills they're missing or which courses to take, in what order. Your system answers that: given a target role and a person's current skills, it finds the skill gap and recommends a sequenced set of courses to close it.

What you will deliver

1. **A Jupyter notebook** that builds *two* recommender approaches (a TF-IDF baseline and an embedding-based one), **compares them**, and **evaluates** them with proper metrics. *This is the graded core.*
2. **An LLM "flourish"** — one call to a free LLM (Groq) that turns the top recommendations into a friendly month-by-month roadmap.
3. **(Optional, only if time allows)** a tiny demo UI that wraps the best model — in-notebook with `ipywidgets`, or as a `streamlit` app.
4. **The Project Report** — the primary assessed artifact (see §07).

The one thing that makes this Data Science, not a chatbot: the recommender — building it, comparing approaches, and measuring how good its recommendations actually are. Everything else supports that.

Scope — read this carefully

✅ IN SCOPE	🚫 OUT OF SCOPE (DELIBERATELY)
Curated course dataset · skill-gap logic · TF-IDF + embedding recommenders · evaluation · charts · one LLM roadmap call · optional tiny demo	Resume-PDF parsing · web scraping · user accounts/login · cloud deployment · a "production" web app

Why so much is out of scope: 7 weeks, remote team, first real project. We are deliberately removing the parts where student projects usually collapse (file parsing, integration, deployment) so you can go *deep* on the data science and finish with a strong report. Ambition lives in the *evaluation*, not in the plumbing.

02

The data (verified, downloadable)

You need two things: a **course catalog** (titles + skill tags) and a **role → required-skills** reference. Use the one dataset below for the catalog, and hand-build the role table. Your Week-1 job is to download the dataset, open it, and confirm the columns — don't assume the schema, verify it.

Course catalog — use this one dataset

Coursera Courses & Skills Dataset 2024 (Kaggle) — Coursera-only, recent, ships a **skills/tags column**. Use the file `coursera_course_dataset_v3.csv` (**623 rows**). On download, run `print(df.columns.tolist())` and rename `Title→title`, `Skills→skills` (Build Spec M1 has the full map). The search text is built from **Title + Skills** — the dataset's `course_description` is only ~65% filled, so we don't rely on it.

Only if that dataset is ever unavailable: any other Kaggle "Coursera / edX / Udemy courses 2024" set works too — but pick one that has a *skills/tags column* and verify it the same way. Don't shop around: one good dataset is enough.

Role → required-skills reference — you build this yourself

Hand-curate a small table for ~8–10 target roles (Data Scientist, Data Analyst, ML Engineer, …), each mapped to the skills it needs. **This is the main path — not a shortcut or a fallback.** It's a few lines of Python (a dictionary), it removes any external dependency, and the roles you choose are part of your report. We've given you a

ready starter — `role_skills_example.py` and `role_skills_table.txt` — keep the roles that fit your dataset, add more, and own the choices.

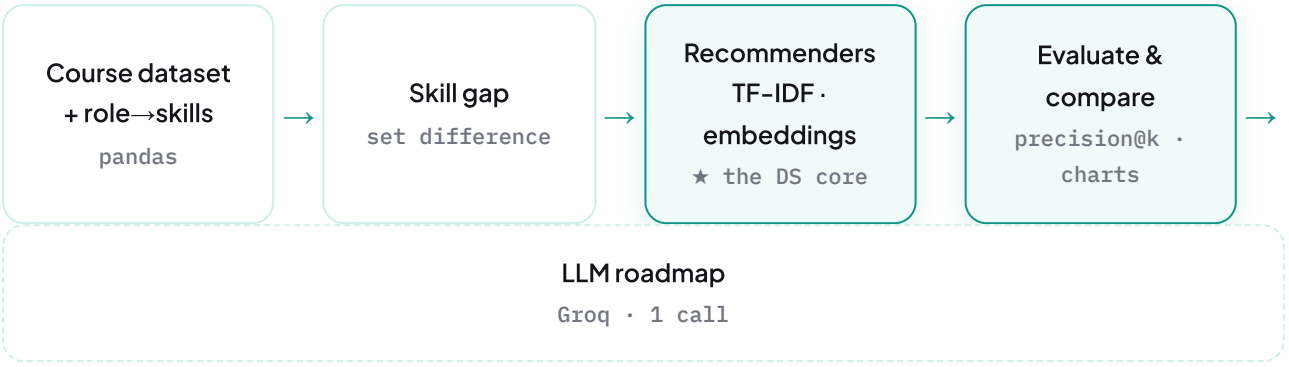
De-risking move: start Week 1 with the hand-built role table + the one course dataset, and get the whole pipeline running end-to-end on small data first. Scale up only once the basic version works.

03

What you're building

Everything below runs **inside one Jupyter notebook** (top to bottom). The optional demo UI can stay in the notebook (`ipywidgets`) or, only if you choose, live outside it as a `streamlit` app.

Your step-by-step build manual: this Handbook explains *what* and *why*. For the exact *how* — every module's inputs and outputs, runnable starter code, worked examples, and common errors — work from the **Build Specification →**. Read it alongside this page.



The same flow, with a real example

Riya knows **Python** and wants to become a **Data Scientist**. Here is what each step produces for her — and where the two real questions get answered: *which other skills does she need?* and *which courses, in what order?*

M1 **Load data** — course catalogue + the role→skills table

```
role_skills["Data Scientist"] = [python, statistics, machine learning, sql,
data viz, deep learning]
courses_raw = 623 real Coursera courses (from Kaggle)
```

↓ courses · role_skills

M2 **Clean & prepare** — build the searchable `text` field

e.g. "SQL for Data Analysis" → text "sql for data analysis query databases joins…" · skills [sql]

↓ courses

M3 **User input + skill gap** — role needs − what Riya has

has [python] · a Data Scientist needs the 6 above
gap = [statistics, machine learning, sql, data visualization, deep learning]

★ Answers Q1 — the "other skills" she needs

↓ query_text (= the gap, as one string)

M4/5 **Two recommenders** — match the gap against every course's `text`

TF-IDF
ranked list

Embeddings
ranked list

↓ tfidf_recs + embed_recs

M6 **Evaluate & pick best** — precision@k chooses the better method

both methods push the gap-relevant courses to the top. TF-IDF and embeddings rank them in a *different order* — the higher-scoring method wins, and its top courses pass to M7. Compare across several roles and k values to see the real difference; there is no single fixed ranking to expect.

★ Answers Q2 — WHICH courses

↓ best_recs

M7 **LLM roadmap (Groq)** — arrange the chosen courses into an order

e.g. "Month 1: SQL · Month 2: Statistics · Months 3–4: Intro to ML · Month 5: Deep Learning · Month 6: Data Viz + capstone" — the LLM's *suggested* order (varies run to run)

★ Answers Q2 — the SEQUENCE

Library map — only two libraries are new to you

COMPONENT	PYTHON LIBRARY	
Data curation & cleaning	`pandas`, `numpy`, built-in `re`	familiar
TF-IDF recommender (baseline)	`scikit-learn` (`TfidfVectorizer`, `cosine_similarity`)	familiar
Embedding recommender	`sentence-transformers` (pulls in `torch`)	new
Evaluation	`scikit-learn`, `pandas`, `numpy`	familiar
Charts	`matplotlib`, `seaborn`	familiar
LLM roadmap	`groq` + `python-dotenv`	new
Optional demo UI	`ipywidgets` (in-notebook) or `streamlit`	optional

The two recommenders, concretely

Baseline — TF-IDF + cosine similarity (verified, standard scikit-learn):

```
# from scikit-learn — turn each course's text (title + skills) into vectors, rank b
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

vec    = TfidfVectorizer(stop_words="english")
matrix = vec.fit_transform(courses["text"])          # every course → a vector
q_vec  = vec.transform([query_text])                 # the skill gap → same space
scores = cosine_similarity(q_vec, matrix).flatten()  # one score per course
```

Second approach — semantic embeddings (from the official sentence-transformers quickstart — runs locally, free):

```
# from sbert.net quickstart
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("all-MiniLM-L6-v2")
course_emb = model.encode(courses["text"].tolist())
q_emb      = model.encode([query_text])
scores     = cosine_similarity(q_emb, course_emb).flatten()
```

You then **compare** the two: which recommends more relevant courses? That comparison is the heart of the report.

The LLM flourish (verified Groq quickstart)

```
# from console.groq.com/docs/quickstart — one call, free tier, no credit card
from groq import Groq
client = Groq(api_key=os.environ["GROQ_API_KEY"])
course_list = "\n".join(f"- {t}" for t in best_recs["title"].head(8))
resp = client.chat.completions.create(
    model="llama-3.3-70b-versatile",
    messages=[{"role":"user","content": f"Turn these courses into a month-by-month
)
print(resp.choices[0].message.content)
```

These snippets show the *shape* of each step (and use the same variables — `courses["text"]`, `query_text`, `best_recs` — throughout). The full, commented, runnable code lives in the **Build Specification**, your build manual.

> The LLM is an **add-on, not load-bearing**. If the Groq call fails on demo day, the recommender study still stands on its own. That is by design.

04

The team — how you'll work together

There are **five of you**. You build **one notebook, together, in order** — Module 0 through Module 7, a module or two at each weekly meeting. Nobody works on a separate copy: whoever is sharing their screen types while the rest follow and help, and the notebook lives in your one shared GitHub repo so there is always a single current version.

The part you *do* divide up is the **report** — writing is naturally a per-person job. Split the chapters so everyone owns a piece:

REPORT CHAPTER	WHAT IT COVERS
Background & Data	the problem, the dataset, how you cleaned it
Methodology	how the two recommenders work (TF-IDF & embeddings), in plain English
Results	precision@k, the comparison, charts — which method won and why
Intro & Conclusion	abstract, introduction, limitations, future work — plus final assembly

Two rules that keep you safe: build the notebook **in order** (each module needs the one before it), and **commit to GitHub every week** so you never lose work. Stuck? Say so early in the chat.

05

Weekly plan & meeting calendar

We meet on Microsoft Teams — the **kickoff is Fri 19 Jun**, then **every Thursday, 7:00–9:00 PM IST** through submission (7 meetings total). Each meeting reviews a concrete deliverable and your blockers. **Come to every meeting with three things:** what you did, what's blocking you, what's next.

The **module codes** below (M0–M8) refer to the build modules in the **Build Specification → Weekly Plan**, which gives the exact build → test-run → demo steps for each sprint. This calendar and that plan are the same schedule, two views.

MEETING	DATE · 7–9 PM IST	WE REVIEW / DECIDE	DELIVERABLE DUE AT THIS MEETING
1 · Kickoff	19 Jun · Fri	Finalize project, roles, setup plan	— (Week-1 work assigned tonight)
2	25 Jun	Is everyone set up? First baseline working?	M0 · M1 · crude M4 **Colab + Groq + GitHub repo** all set up · 2 candidate datasets' columns reported · a crude TF-IDF recommender returns *something*
3	2 Jul	Baseline + embeddings	M2 · M3 · M4 · M5 Clean TF-IDF recommender + skill-gap logic · embedding recommender running
4	9 Jul	Evaluation & comparison	M6 Evaluation harness (precision@k) · first comparison table + charts
5	16 Jul	LLM step + scope freeze	M7 M8? Groq roadmap cell working · **feature freeze** · decision on optional demo
6	23 Jul	Report draft	optional M8 report Each owner's draft section · final charts/tables · notebook cleaned
7 · Final	30 Jul	Last review before submission	Full assembled report draft · dry-run walkthrough
Submit	31 Jul (Fri)	—	Final report submitted to ISI

Timeline reality: kickoff (Fri 19 Jun) to deadline (31 Jul) is ≈ 6 working weeks across 7 meetings (kickoff on a Friday, then weekly Thursdays). It's tight — that's exactly why scope is kept lean. No new ideas after Meeting 5's feature freeze.

06

Reading list

Grouped by topic. Start with the two new areas — that's where your learning gap is. Everything links to **official docs or established tutorials** (all checked).

Recommender systems — the concept

- DataCamp — Recommender Systems in Python (content-based & collaborative)
- "Recommender Systems: A Primer" (arXiv survey) — for the report's background section

TF-IDF + cosine similarity familiar

- scikit-learn — Text feature extraction (TF-IDF)

Embeddings new

- Sentence-Transformers — Quickstart
- Sentence-Transformers — Computing Embeddings

Evaluation

- scikit-learn — Model evaluation (precision/recall; *precision@k for ranking is usually computed manually — your mentor will walk you through it*)

LLM API new

- Groq — Quickstart · groq-python library · Groq API Cookbook

Tools familiar

- **Google Colab** — Welcome to Colaboratory (Google's own intro notebook) · beginner walk-through
- Streamlit docs (only if you build the optional demo as a Streamlit app)

07

The Project Report

The report is the **primary assessed deliverable** — and you should write it *as you go*, not in the last week.

Format pending from ISI. ISI uses its own report format, which we haven't received yet. The official template will be confirmed in **Meeting 2 (25 Jun)**. Until then, use the provisional outline below only as a guide for *what content to capture* — not the final structure.

Provisional outline (start gathering this content now): Abstract · Introduction · Background (recommenders, TF-IDF vs embeddings) · Data · Methodology · Experiments & Evaluation *(your strongest section)* · Results & Discussion · Limitations & Future Work · Conclusion · References · Appendix (the notebook).

Divide the chapters across the team (see §04) so everyone owns a piece — e.g. Background & Data · Methodology · Evaluation & Results · Abstract, Introduction & Conclusion · final assembly. Agree who takes what in Meeting 2.

08

Tools, setup & where things live

Where everything lives

TOOL	USED FOR
Microsoft Teams (our group community)	**All communication, the weekly meetings (Teams video), and file sharing** — datasets, slides, the report, meeting notes. This is our home base.
GitHub (one shared repo)	**All code** — the notebook + `requirements.txt`. The team **creates one repo in Week 1** and commits to it every week.
Google Colab	The platform you **build on** — a free, browser-based Jupyter notebook. Nothing to install.

Join the Teams community first: teams.live.com/l/community/FEAZwyZnE0UtNev6g — everyone joins before Meeting 1 (Fri 19 Jun).

New to Google Colab? Start here

- Welcome to Colaboratory — Google's own intro notebook (it opens automatically when you visit Colab).

- Getting Started with Google Colab — GeeksforGeeks — a short written walk-through.

Colab is just Jupyter in the browser — the same kind of notebooks you used in the ML/DL sessions, but with nothing to install and an identical environment for all five of you. `pandas`, `numpy` and `matplotlib` come pre-installed; you'll `pip install sentence-transformers` and `groq` in a cell.

Day-1 checklist (do all of this in Week 1)

1. **Join the Teams community** (link above) — everyone.
2. **Google Colab** — everyone signs in with a Google account at colab.research.google.com.
3. **Create ONE shared GitHub repo** for the team, add a `requirements.txt`, give all five access. (You learned GitHub in the course.) **All code lives here.**
4. **Free Groq key** — sign up at console.groq.com (no credit card), create an API key, store it as a Colab secret / in a `.env` file. Never paste keys into shared code or GitHub.
5. **Smoke test** — one Colab cell that (a) imports pandas, (b) imports `sentence-transformers`, (c) makes one Groq call. All three work → environment ready.

> **Security:** a leaked API key can be abused. Keep keys out of GitHub — use Colab Secrets or a `.gitignore`d `.env` file.

09

How we work together

- **Weekly 2-hour touchpoint with the mentor.** Come prepared: what you did, what's blocking you, what you'll do next. The agenda for each week is in §05.
- **Definition of "done" each week** = code committed to GitHub + a one-paragraph note of progress + blockers named. "Almost working" on a laptop doesn't count — push it.
- **Say when you're stuck.** A blocker raised on day 2 is a 10-minute fix; the same blocker hidden until the touchpoint costs a week. There is no penalty for asking.
- **Be honest in the report.** "The embedding approach didn't beat TF-IDF and here's why" is a *strong* result, not a failure. Examiners reward honest evaluation over

inflated claims.

Remember the goal: a working notebook you understand deeply + a report that honestly evaluates it. Not a flashy app. Depth beats shine.
